
ULTRALYTICS YOLO EVOLUTION: AN OVERVIEW OF YOLO26, YOLO11, YOLOv8, AND YOLOv5 OBJECT DETECTORS FOR COMPUTER VISION AND PATTERN RECOGNITION

 **Ranjan Sapkota**  **Manoj Karkee**

Cornell University, Biological & Environmental Engineering, Ithaca, NY 14850, USA
rs2672@cornell.edu

March 17, 2026

ABSTRACT

This paper presents a comprehensive overview of the Ultralytics YOLO family of object detectors, emphasizing the architectural evolution, benchmarking, deployment perspectives, and future challenges. The review begins with the most recent release, YOLO26 (or YOLOv26), which introduces key innovations including Distribution Focal Loss (DFL) removal, native NMS-free inference, Progressive Loss Balancing (ProgLoss), Small-Target-Aware Label Assignment (STAL), and the MuSGD optimizer for stable training. The progression is then traced through YOLO11, with its hybrid task assignment and efficiency-focused modules; YOLOv8, which advanced with a decoupled detection head and anchor-free predictions; and YOLOv5, which established the modular PyTorch foundation that enabled modern YOLO development. Benchmarking on the MS COCO dataset provides a detailed quantitative comparison of YOLOv5, YOLOv8, YOLO11, and YOLO26, alongside cross-comparisons with YOLOv12, YOLOv13, RT-DETR, and DEIM. Metrics including precision, recall, F1 score, mean Average Precision, and inference speed are analyzed to highlight trade-offs between accuracy and efficiency. Deployment and application perspectives are further discussed, covering export formats, quantization strategies, and real-world use in robotics, agriculture, surveillance, and manufacturing. Finally, the paper identifies challenges and future directions, including dense-scene limitations, hybrid CNN–Transformer integration, open-vocabulary detection, and edge-aware training approaches.

Keywords YOLO · Ultralytics · YOLOv5 · YOLOv8 · YOLO11 · YOLO26 · You Only Look Once

1 Introduction

Object detection has emerged as one of the most critical tasks in computer vision, enabling machines not only to identify but also to localize multiple objects within complex image or video streams [1, 2]. Its importance spans a wide spectrum of domains, including autonomous driving, robotics, surveillance, medical imaging, agriculture, and smart manufacturing, where reliable real-time performance directly translates into safety, efficiency, and automation gains [3, 4]. Among the numerous algorithms proposed over the past decade, the *You Only Look Once* (YOLO) family has emerged as the most influential and widely adopted series of models for real-time object detection, striking a balance between high accuracy and unprecedented inference speed [5]. Since its initial release in 2016, YOLO has undergone multiple architectural revisions, each iteration addressing specific limitations of its predecessors while incorporating advances in neural network design, training strategies, loss functions, and deployment efficiency [5]. The cumulative evolution of YOLO is visualized in Figure 1, which traces the transition from early Ultralytics releases such as YOLOv5 to the latest YOLO26 (2025), the first version to natively unify five key tasks: object detection, instance segmentation, classification, pose/keypoints detection, and oriented bounding box detection. This timeline illustrates how YOLO has steadily expanded its capabilities beyond detection to become a versatile multi-task vision framework suitable for both research and edge deployment.

The release of YOLO26 in September 2025 represents the latest milestone in this evolutionary trajectory. Designed around principles of simplicity, efficiency, and innovation, YOLO26 introduces architectural simplifications that remove bottlenecks such as Non-Maximum Suppression (NMS) and Distribution Focal Loss (DFL), adopts a novel MuSGD optimizer for stable convergence, and incorporates training refinements such as Progressive Loss Balancing (ProgLoss) and Small-Target-Aware Label Assignment (STAL) [6]. Together, these innovations establish YOLO26 as a cutting-edge model optimized for low-power and embedded devices, significantly improving real-world deployability without compromising accuracy.

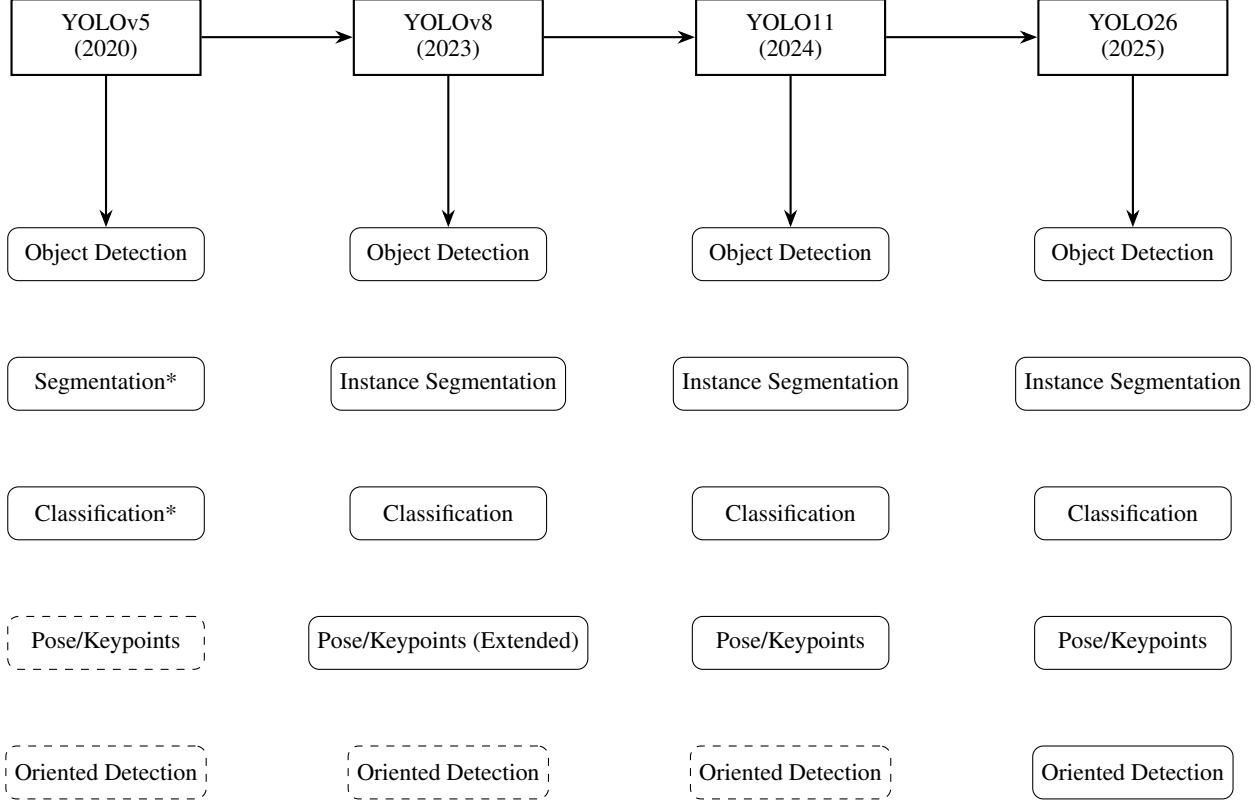


Figure 1: Timeline of Ultralytics YOLO models (YOLOv5,11 and YOLOv8, YOLO11 and YOLO26) and their task support. Solid boxes = supported natively, dashed boxes = not supported. * indicates features added later via community extensions.

To situate YOLO26 within the broader ecosystem, we first recall the paradigm shift inaugurated by Redmon *et al.* (2016), who reframed detection as a single regression problem eschewing region proposals in favor of direct, one-shot prediction of bounding boxes and class probabilities [7]. In contrast to two-stage pipelines such as R-CNN and Faster R-CNN, which decouple proposal generation and classification [8, 9], the YOLO design achieved real-time throughput while preserving competitive accuracy, making YOLOv1 attractive for latency-critical domains including robotics and autonomous navigation [10, 11]. Rapid iterations followed: YOLOv2 (2017) introduced batch normalization, anchor boxes, and multi-scale training for robustness across object scales [12]; YOLOv3 (2018) deepened the backbone (Darknet-53) and exploited multi-scale feature maps, establishing a widely adopted baseline for both academia and industry [13, 14, 5, 15]. As accuracy demands grew across aerial imaging, agriculture, and medical analysis, architectures diversified. YOLOv4 (2020) incorporated CSPNet, improved activations, and advanced training strategies (e.g., mosaic augmentation, CIoU) [16]. Ultralytics’ YOLOv5 (2020) popularized a PyTorch-native, modular toolchain that eased adaptation to segmentation, classification, and edge deployment. Subsequent community releases (YOLOv6, YOLOv7) integrated parameter-efficient modules and transformer-inspired blocks to push accuracy while maintaining real-time inference [17, 18]. Ultralytics then re-architected the stack with YOLOv8 (decoupled head, anchor-free predictions) [19, 20], followed by YOLOv9 (GELAN, progressive distillation) [21], YOLOv10 (latency-balanced assignments) [22], and YOLO11 (efficiency with strong small-object performance) [5]. Parallel non-Ultralytics lines YOLOv12 and YOLOv13 pursued attention-centric designs (multi-head self-attention, improved fusion, stronger regularization) but continued to rely on NMS and DFL, creating latency and export friction on low-power devices [23, 24]. Addressing these bottlenecks, YOLO26 advances a deployment-first philosophy: it removes DFL, adopts

end-to-end (NMS-free) inference, and introduces ProgLoss and STAL for stability and small-object fidelity, while training is accelerated via MuSGD. Figure 1 summarizes this trajectory and clarifies YOLO26 as the first Ultralytics release that natively unifies object detection, instance segmentation, classification, pose/keypoints detection, and oriented bounding box detection.

This paper aims to provide a consolidated understanding of how Ultralytics has shaped the YOLO family into its current form, culminating in YOLO26. By systematically comparing these four landmark versions, this review not only underscores architectural innovations but also contextualizes performance trade-offs, benchmarking outcomes, and deployment-readiness. In doing so, it addresses the broader narrative of YOLO’s progression from a fast but limited detector into a versatile, multi-task, edge-optimized framework that continues to set the standard for real-time object detection in both research and practice.

2 Architectural Evolution of Ultralytics YOLO Models

To provide a precise chronological and architectural context, we separate Ultralytics-maintained releases from community-driven variants. *Table 1* catalogs the Ultralytics lineage YOLOv5 (2020), YOLOv8 (2023), YOLO11 (2024), and YOLO26 (2025) highlighting design choices, capabilities, and deployment characteristics within a single vendor family. In contrast, *Table 2* surveys major community-driven releases independent of Ultralytics, including YOLOv1 (2015), YOLOv2 (2016), YOLOv3 (2018), YOLOv4 (2020), YOLOv6 (2022), YOLOv7 (2022), YOLOv9 (2024), and YOLOv10 (2024), alongside other contemporary variants. Together, these tables present a coherent, year-by-year view of architectural innovations, task expansions, and performance trends across the YOLO ecosystem, clarifying how the framework matured into the state represented by YOLO26 (Tables 1-2).

Table 1: Ultralytics YOLO models: key architectural innovations, tasks, and frameworks

Model (Year)	Key Architectural Innovation and Contribution	Tasks	Framework
YOLOv5 (2020) (Source Link)	First PyTorch implementation by Ultralytics, replacing Darknet; introduced SiLU activation and PANet neck for improved feature aggregation; flexible anchor-free head; made YOLO accessible with modern training utilities, augmentations, and export options.	Object Detection, Limited Instance Segmentation	PyTorch
YOLOv8 (2023) (Source Link)	Next-gen Ultralytics redesign: C2f backbone for lightweight representation, decoupled detection head for improved convergence, fully anchor-free design; introduced task unification across detection, segmentation, pose/keypoints, and panoptic tasks; strong open-source ecosystem integration.	Object Detection, Instance Segmentation, Panoptic Segmentation, Keypoint Estimation	PyTorch
YOLO11 (2024) (Source Link)	Added C3k2 CSP bottlenecks (smaller kernel CSP blocks) Major Ultralytics milestone: added C3k2 CSP bottlenecks for efficiency, C2PSA module (CSP + spatial attention) for robust feature focus; extended YOLO family beyond detection/segmentation to include pose estimation and oriented bounding boxes.	Object Detection, Instance Segmentation, Pose Estimation, Oriented Detection	PyTorch
YOLO26 (2025) (Source Link)	Edge-optimized flagship: eliminated NMS with native end-to-end predictor, removed DFL for faster exports and simpler regression; introduced ProgLoss (progressive loss balancing) and STAL (small-target-aware label assignment) for stability and small-object accuracy; integrated MuSGD optimizer for LLM-inspired stable convergence; up to 43% faster inference on CPUs and Jetson devices.	Object Detection, Instance Segmentation, Pose Estimation, Oriented Detection, Classification	PyTorch

Non-Ultralytics YOLO versions:

Table 2: Other YOLO models(non-ultralytics): key architectural innovations, tasks, and frameworks

Model (Year)	Key Architectural Innovation and Contribution	Tasks	Framework
YOLOv1 (2015) [7]	First unified single-stage detector; predicted bounding boxes + class probabilities in a single forward pass, pioneering real-time object detection.	Object Detection, Classification	Darknet
YOLOv2 (2016) [12]	Introduced anchor boxes and dimension clustering; enabled multi-scale training; YOLO9000 demonstrated joint detection and classification across 9000 categories.	Object Detection, Classification	Darknet
YOLOv3 (2018) [13]	Deeper Darknet-53 backbone with residual blocks; multi-scale predictions enhanced small-object detection; added SPP for richer feature context.	Object Detection, Multi-scale Detection	Darknet
YOLOv4 (2020) [16]	CSPDarknet-53 backbone and Mish activation; improved training stability; optimized for high accuracy-speed tradeoff on GPUs.	Object Detection, Object Tracking	Darknet
YOLOv6 (2022) [17]	Industrial-focused release (by Meituan): EfficientRep backbone, improved self-attention, anchor-free head; focused on deployment efficiency for production.	Object Detection, Instance Segmentation	PyTorch
YOLOv7 (2022) [18]	Introduced E-ELAN for extended feature learning; re-parameterized convolution modules; incorporated transformer blocks for context modeling; optimized for tracking and multi-task vision.	Object Detection, Tracking, Segmentation	PyTorch
YOLOv9 (2024) [21]	Introduced Programmable Gradient Information (PGI) for enhanced learning and G-ELAN backbone; boosted training flexibility and feature richness.	Object Detection, Instance Segmentation	PyTorch
YOLOv10 (2024) [22]	Achieved end-to-end NMS-free detection using consistent dual-assignment training strategy; removed reliance on post-processing.	Object Detection	PyTorch
YOLOv12 (2025) [23]	Attention-centric design: efficient area attention module and Residual ELAN (R-ELAN) blocks; achieved transformer-level accuracy at YOLO speed.	Object Detection	PyTorch
YOLOv13 (2025) [24]	Proposed HyperACE (hypergraph-based correlation enhancement) and FullPAD (pipeline-wide aggregation/distribution) for global high-order feature learning; depth-wise separable convolutions improved efficiency.	Object Detection	PyTorch

Ultralytics-maintained YOLO releases illustrate a steady trajectory toward modularity, task unification, and deployment efficiency, culminating in YOLO26 as the first fully integrated framework for detection, segmentation, pose estimation, oriented bounding boxes, and classification. In contrast, community-driven models have often prioritized experimental architectural features ranging from Darknet-based foundations in YOLOv1–YOLOv4 to transformer-inspired mechanisms and attention modules in YOLOv6–YOLOv13. While these variants advanced accuracy benchmarks and introduced novel ideas such as GELAN or hypergraph-based feature aggregation, they frequently retained computationally demanding components such as NMS or DFL, which limited seamless edge deployment. Ultralytics models, by contrast, consistently emphasize exportability, quantization, and production-ready APIs, reflecting a pragmatic design philosophy that complements rather than competes with academic prototypes. The comparative overview in Table 2 underscores how Ultralytics’ streamlined design choices have positioned YOLO26 as both a research-relevant and industry-ready milestone within the broader YOLO ecosystem.

2.1 YOLO26: DFL Removal, NMS-Free Inference, ProgLoss, STAL, MuSGD

YOLO26 (2025) is an edge-first redesign that codifies end-to-end simplicity and export robustness. Two decisive architectural changes lead this release. First, the head removes Distribution Focal Loss (DFL)–based distributional regression in favor of a lighter, hardware-friendly parameterization of bounding boxes. Eliminating DFL prunes operators that were brittle across compilers and runtimes, reducing graph complexity and easing quantization. Second, the decoding path is reworked for **NMS-free, end-to-end inference**: the head produces a compact, non-redundant set of predictions without a post-processing suppression step. This erases a traditional latency bottleneck and removes deployment-time hyperparameters (IoU/score thresholds) that often demand scenario-specific tuning.

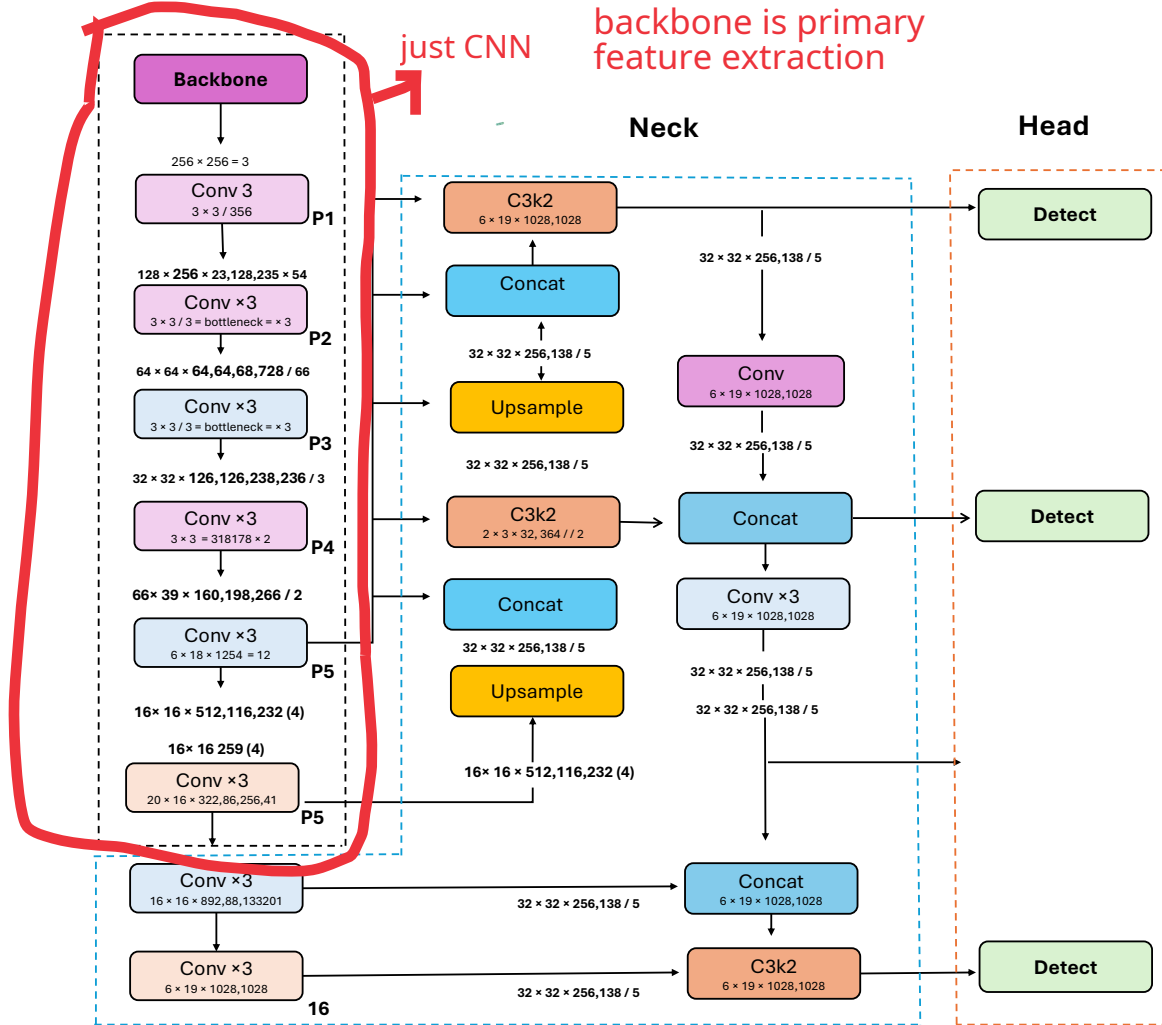


Figure 2: YOLO26 core architecture diagram

Training dynamics are stabilized by **ProgLoss** (progressive loss balancing) and **STAL** (small-target-aware label assignment). ProgLoss schedules the relative weights of classification, localization, and auxiliary terms to prevent domination by easy negatives or large objects in later epochs, improving convergence smoothness. STAL adjusts assignment priors and spatial tolerance to ensure tiny, occluded, or low-contrast instances receive adequate supervisory signal, boosting recall in edge-centric imagery (UAV, smart cameras, mobile robotics). Optimization is driven by **MuSGD**, a hybrid that combines the generalization and simplicity of SGD with curvature- and momentum-aware updates inspired by modern large-model training. Empirically, MuSGD shortens time-to-quality and mitigates late-epoch oscillations, aiding reproducible convergence across scales (n/s/m/l/x).

On the systems side, YOLOv26 leans into **portability and quantization**. By removing DFL and NMS, the exported graphs map cleanly to ONNX, TensorRT, CoreML, and TFLite with fewer custom kernels, facilitating INT8 and FP16 deployment with minimal accuracy loss. The simplified operator surface reduces conversion friction and improves determinism across compilers. Latency benefits accrue on CPUs and embedded GPUs, with notable gains at batch size one where post-processing previously dominated. With unified support for detection, instance segmentation, classification, pose/keypoints, and oriented bounding boxes, YOLOv26 consolidates the family into a single, multi-task framework. In effect, it operationalizes a long-standing objective for real-time detection: align state-of-the-art accuracy with a *minimal* and *export-friendly* graph that scales from cloud GPUs to resource-constrained edge devices.

YOLOv26 architecture (Figure 2) consolidates deployability and accuracy with two core simplifications: removing Distribution Focal Loss to ease export/hardware compatibility and enabling native end-to-end (NMS-free) inference to eliminate post-processing latency. Training stability and small-object recall are strengthened via Progressive Loss Balancing and Small-Target-Aware Label assignment, while MuSGD blends SGD with Muon-style curvature/momentum for fast, smooth convergence. Early previews indicate substantial CPU-side speedups (approximately 43%) and consistent performance under FP16/INT8 quantization, making the family well suited for edge devices (Jetson/CPU) alongside standard GPU pipelines.

2.2 YOLOv11: Hybrid Task Assignment and Efficiency

YOLOv11 (2024) focused on balancing accuracy, stability, and device efficiency with refinements that touched both representation and supervision. On the representation side, compact CSP variants (e.g., C3/C3k2 bottlenecks) and lightweight attention modules were emphasized to increase channel interaction without inflating latency. These changes improved feature reuse and reduced redundancy, yielding better FLOPs-to-mAP ratios on mainstream GPUs and edge SoCs. Neck design continued to target low-loss multi-scale fusion with careful stride harmonization, limiting aliasing and improving localization at small scales.

The supervisory signal adopted a hybrid, task-aware assignment: label assignment and loss weighting were tuned jointly across classification, localization, and (where enabled) auxiliary tasks such as segmentation or pose. This hybridization reduced failure modes in crowded scenes and improved recall at fixed precision by stabilizing which features are optimized for which objective at each training stage. Importantly, the assignment strategy was designed to be robust to batch-size variability, an increasingly common constraint in edge training and fine-tuning scenarios.

From a system perspective, YOLOv11 strengthened multi-task support in the Ultralytics stack object detection, instance segmentation, classification, pose estimation, and (in supported builds) oriented bounding boxes while preserving the same export surfaces introduced in prior releases. It retained NMS-based decoding but reduced post-processing load through crisper, less redundant predictions. Together, the efficiency gains and hybrid task assignment positioned YOLOv11 as a versatile mid-generation release: more accurate than YOLOv8 under similar latency budgets, and more stable under transfer and domain shift, particularly for small-object and moderately dense scenes.

YOLOv11 architecture as illustrated in Figure 3 replaced C2f with compact C3k2 blocks, retained SPPF for multi-scale features, and introduced C2PSA to enhance spatial attention, yielding a leaner backbone/neck with improved feature selectivity. Empirical summaries note fewer parameters than comparable YOLOv8 variants and modest inference speed gains over YOLOv10, with support for a wide task set (detection, segmentation, pose, classification, OBB). The design goal was to preserve throughput while boosting representation in cluttered, small-object scenes across edge and GPU stacks.

2.3 YOLOv8: Decoupled Head, Anchor-Free Predictions

YOLOv8 delivered a more principled detector head and a modernized representation pipeline. The decoupled head separates classification and regression branches, mitigating gradient interference between localization and recognition objectives [25]. This design reduces optimization entanglement improving convergence smoothness and yielding better calibration at a given operating point (e.g., at fixed mAP or recall). Complementing this head, YOLOv8 formalized

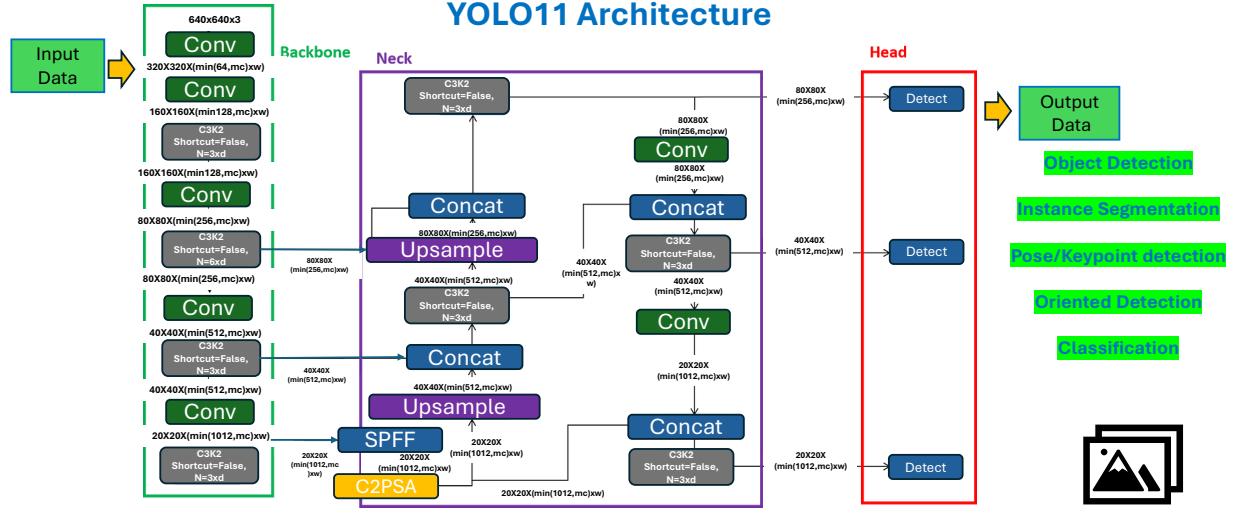


Figure 3: YOLO11 architecture diagram

an anchor-free assignment strategy: instead of regressing offsets around predefined anchor priors, the model predicts normalized box parameters and confidences directly for candidate points on the feature map [25, 26, 27]. Anchor-free heads simplify hyperparameter tuning (no anchor clustering) and improve generalization across datasets with differing object aspect-ratio statistics [28, 29].

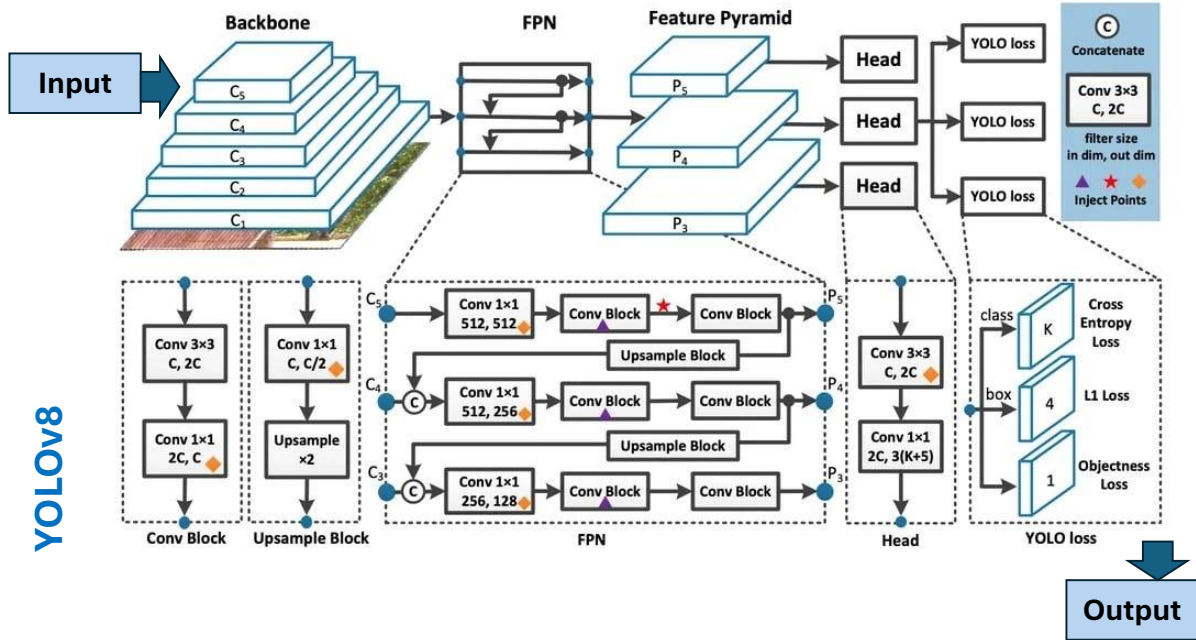


Figure 4: YOLOv8 architecture diagram

The backbone/neck stack evolved toward lighter, high-throughput modules (e.g., C2f-style blocks) that maintain receptive-field richness while reducing memory bandwidth pressure [30, 31]. Multi-scale features are fused with attention to stride alignment and aliasing minimization, preserving crisp spatial information critical to small objects. Training refinements included improved label assignment heuristics, class-imbalance mitigation, and stronger regularization

under aggressive augmentation, which together stabilized long-horizon training and reduced sensitivity to batch size. These choices enabled coherent scaling across nano to extra-large variants without pathological underfitting or collapse at the smallest scales.

Operationally, YOLOv8 expanded first-class support beyond detection to instance segmentation, classification, and (via an extended head) keypoints/pose. The PyTorch-first design continued to emphasize exportability (ONNX, TensorRT, CoreML, TFLite) and FP16 inference, further smoothing the path from research to deployment. While YOLOv8 still used conventional NMS and (in many settings) distributional regression for boxes, its head decoupling and anchor-free philosophy foreshadowed the end-to-end direction later realized in YOLOv26.

The YOLOv8 architecture (Figure 4) improved convergence and small-object fidelity, complemented by CIOU/DFL losses and an expanded task portfolio (detection, segmentation, pose, tracking, classification). Reported results highlight strong COCO AP with high throughput on TensorRT, and multiple model scales (n–x) for flexible deployment. Collectively, these choices advanced accuracy without sacrificing real-time performance.

2.4 YOLOv5: Modularity and PyTorch Adoption

Ultralytics’ YOLOv5 marked a decisive shift from Darknet to a fully PyTorch-native implementation, catalyzing adoption by unifying training utilities, data pipelines, augmentation strategies, and export tooling under a single, accessible codebase [32]. Architecturally, YOLOv5 popularized a family of depth/width–scaled variants (n/s/m/l/x) built on CSP-inspired backbones and PANet-style necks to improve multi-scale aggregation with minimal latency overhead [33, 34]. The transition to SiLU activations and careful normalization choices reduced optimization pathologies (e.g., vanishing gradients in deeper stacks) while preserving high-frequency detail important for small-object detection [35].

A key contribution of YOLOv5 is modularity: components (backbone blocks, neck layers, heads, losses) can be swapped, ablated, or extended with minimal refactoring [36]. This modularity underpins rapid task adaptation e.g., adding a segmentation head or a classification head without destabilizing the base detector. On the training side, YOLOv5 systematized strong, production-friendly augmentations (mosaic, copy-paste, HSV jitter, random perspective) and reliable schedules (cosine, one-cycle), paired with EMA weight tracking for robust final checkpoints. The data loader and caching logic were engineered for throughput on commodity hardware, enabling high iteration rates that accelerate experimentation cycles.

From a deployment perspective, YOLOv5 leaned into broad format support (ONNX, TensorRT, CoreML), mixed-precision inference (FP16), and lightweight post-processing [37, 38, 39]. Although it retained NMS and conventional bounding-box regression (with optional DFL in later forks), the streamlined graph and conservative operator set made it particularly amenable to embedded and cloud accelerators. Collectively, these design choices positioned YOLOv5 as a practical baseline: not the most novel architecturally, but extremely effective in closing the gap between research-grade code and production-grade inference at scale.

The YOLOv5 architecture (Figure 5) release prioritized reproducibility, streamlined augmentation pipelines, and frictionless export, which accelerated adoption in research and production. The emphasis on developer ergonomics and deployment breadth rather than radical new blocks positioned YOLOv5 as a durable baseline that subsequent versions iterated upon for accuracy/latency trade-offs.

3 Benchmarking and Comparative Analysis

The performance of real-time object detectors is typically measured on the MS COCO dataset, where the mean Average Precision (mAP) across multiple intersection-over-union thresholds (0.50 to 0.95) quantifies detection quality, and per-image inference latency assesses suitability for edge deployment. This section compares four Ultralytics releases YOLOv5, YOLOv8, YOLO11, and YOLO26 and situates them within the broader landscape by contrasting with YOLOv12, YOLOv13, RT-DETR variants, and the DEIM training framework. Three subsections examine: (i) quantitative evaluation of Ultralytics models on MS COCO; (ii) cross-comparisons with recent transformer-oriented detectors; and (iii) the metrics used for these evaluations.

3.1 Quantitative evaluation of YOLOv5, YOLOv8, YOLO11, and YOLO26

The Ultralytics lineage exhibits steady gains in accuracy and efficiency across successive releases (Refer to Table 3). **YOLOv5** (2020) introduced a modular PyTorch implementation supporting depth/width-scaled variants (n, s, m, l, x). The SiLU-activated backbone and PANet neck yield competitive accuracy; however, reliance on Non-Maximum Suppression (NMS) and, in some configurations, Distribution Focal Loss (DFL) adds post-processing overhead. On

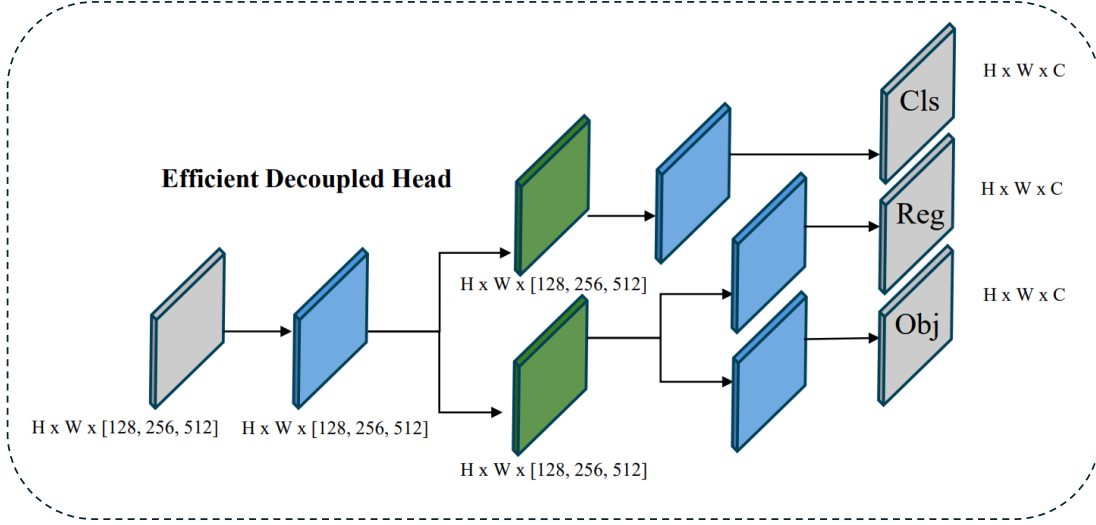


Figure 5: YOLOv5 architecture diagram

COCO val, YOLOv5u-n (640 px) reports $\text{mAP} \approx 34.3\%$ at ~ 73.6 ms per image on CPU (ONNX), while YOLOv5u-s reaches $\approx 43.0\%$ mAP at ~ 120.7 ms. These figures provide a practical baseline for later releases.

YOLOv8 (2023) improves convergence and small-object fidelity via a decoupled head and anchor-free design. A C2f backbone with SPPF reduces parameters while preserving receptive-field richness. YOLOv8n (640 px) delivers $\approx 37.3\%$ mAP at ~ 80.4 ms per image on CPU, with larger variants (s, m, l) trading latency for accuracy (e.g., YOLOv8s $\approx 44.9\%$ mAP at ~ 128.4 ms; YOLOv8m $\approx 50.5\%$ mAP at ~ 197.5 ms). Detection, instance segmentation, and classification are first-class tasks, though final predictions remain NMS-based.

YOLO11 (2024) targets efficiency and small-object performance with compact C3k2 bottlenecks and a C2PSA attention module for robust feature aggregation. YOLO11n (640 px) attains $\approx 39.5\%$ mAP with lower CPU latency (~ 56.1 ms) than YOLOv5u-n, while YOLO11s and YOLO11m achieve $\approx 47.0\%$ and $\approx 50.3\%$ mAP at ~ 90.0 ms and ~ 171.0 ms, respectively. The family extends support to pose estimation and oriented bounding boxes, yet still employs NMS at inference.

YOLO26 (2025) implements the most radical simplifications: removal of DFL and native end-to-end (NMS-free) inference, alongside Progressive Loss Balancing (ProgLoss), Small-Target-Aware Label Assignment (STAL), and a MuSGD optimizer for stable, fast convergence. YOLO26n (640 px) reports $\approx 39.8\%$ mAP (up to $\approx 40.3\%$ in end-to-end mode) at ~ 38.9 ms on CPU substantially faster than YOLO11n at comparable accuracy. YOLO26s and YOLO26m reach $\approx 47.2\%$ and $\approx 51.5\%$ mAP at ~ 87.2 ms and ~ 220.0 ms; YOLO26l improves to ≈ 53.0 – 53.4% mAP at ~ 286.2 ms. These results underscore a deployment-centric design: by eliminating NMS and DFL, CPU inference improves markedly while preserving accuracy across scales.

Table 3: Detection performance of Ultralytics YOLO models on MS COCO validation. Latency is milliseconds per image on CPU (ONNX) and T4 GPU (TensorRT). Variants: n (nano), s (small), m (medium), l (large).

Model	Variant	mAP50–95 (%)	CPU ONNX (ms)	T4 TensorRT (ms)	Params (M)	FLOPs (B)
YOLOv5u	n	34.3	73.6	1.06	2.6	7.7
	s	43.0	120.7	1.78	7.2	17.0
YOLOv8	n	37.3	80.4	0.99	3.2	8.7
	s	44.9	128.4	1.90	11.2	25.9
	m	50.5	197.5	3.30	25.9	63.3
YOLO11	n	39.5	56.1	1.5	2.6	6.5
	s	47.0	90.0	1.9	9.2	16.7
	m	50.3	171.0	3.0	20.0	44.8
	l	52.2	308.7	4.8	24.8	53.4
YOLO26	n	39.8 (40.3 e2e)	38.9	1.7	2.4	5.4
	s	47.2 (47.6 e2e)	87.2	2.7	9.5	20.7
	m	51.5 (51.7 e2e)	220.0	4.9	20.4	68.2
	l	53.0–53.4	286.2	6.5	24.8	86.4

3.2 Cross-comparisons with YOLOv12, YOLOv13, RT-DETR, and DEIM

Outside the Ultralytics line, several detectors pursue higher accuracy via attention and transformer mechanisms (Refer to Table 4). **YOLOv12** introduces efficient area attention and R-ELAN blocks, publishing a spectrum of COCO results from the nano through large variants (e.g., YOLO12n $\sim 40.6\%$ AP at ~ 1.64 ms on T4; YOLO12s $\sim 48.0\%$ at ~ 2.61 ms; YOLO12m $\sim 51.9\%$ at ~ 5.73 ms; YOLO12l $\sim 53.2\%$ at ~ 6.14 ms). **YOLOv13** advances global context modeling with hypergraph correlation and pipeline-wide aggregation, reporting nano through large tiers that close the gap with transformer detectors (e.g., small and large variants at $\sim 52\%$ and $\gtrsim 56\%$ AP with mid-tens of milliseconds on T4). These families emphasize richer feature interactions while retaining convolutional efficiency.

Transformer-style, end-to-end detectors provide a complementary perspective. **RT-DETR** replaces NMS with bipartite matching, achieving ~ 51 AP on COCO within a real-time latency envelope on T4; iterative refinements in RT-DETRv2 and RT-DETRv3 push accuracy into the mid-50s AP at modest increases in latency. **DEIM** (a training framework) further elevates DETR-like systems; combined with D-FINE, published results include AP ~ 54.7 at ~ 124 fps (L) and AP ~ 56.5 at ~ 78 fps (X) on T4. While these transformer pipelines achieve the highest AP, they typically incur higher computational complexity and are less favorable for CPU-bound edge scenarios. In contrast, YOLO26 prioritizes export simplicity, quantization robustness, and NMS-free decoding to reduce latency variance and integration effort on heterogeneous hardware.

Table 4: Comparative MS COCO benchmarking for recent non-Ultralytics YOLO variants and transformer-style detectors. AP is mAP@50–95; latency is per-image on an NVIDIA T4 (TensorRT where applicable).

Model	Variant	AP (%)	Latency (ms)	Notes
YOLOv12	n / s / m / l	40.6 / 48.0 / 51.9 / 53.2	1.64 / 2.61 / 5.73 / 6.14	Efficient area attention, R-ELAN
YOLOv13	n / s / l	~ 42 / ~ 52 / $\gtrsim 56$	~ 41 / ~ 48 / ~ 67	Hypergraph correlation, pipeline aggregation
RT-DETR	base	~ 51	~ 46	End-to-end bipartite matching
RT-DETRv2	base	~ 52.7	~ 48	Enhanced fusion and decoding
RT-DETRv3	base	~ 54.7	~ 54	Further architectural refinements
DEIM + RT-DETRv2	base	~ 53.2		Training framework improves AP
DEIM-D-FINE	L / X	~ 54.7 / ~ 56.5	~ 8.1 / ~ 12.8	~ 124 fps / ~ 78 fps (T4)

3.3 Metrics: precision, recall, F1, mAP, inference speed

Quantitative evaluation relies on complementary criteria. **Precision** measures the fraction of predicted boxes that match ground truth, while **recall** quantifies the fraction of ground-truth objects that are detected. The **F1 score** the harmonic mean of precision and recall summarizes the trade-off between false positives and false negatives. In object detection, the field predominantly reports **mean Average Precision (mAP)**, computed by integrating precision–recall curves across multiple IoU thresholds and classes. MS COCO averages AP at IoUs 0.50:0.05:0.95 (AP50–95), a stringent measure that is sensitive to localization accuracy and class balance.

The interplay between accuracy and latency is central to model selection. Smaller models (e.g., nano and small variants) offer favorable throughput on CPUs and embedded GPUs, making them attractive for robotics, UAVs, and smart cameras; larger variants maximize mAP at the expense of inference time, which can be acceptable for server-side analytics. End-to-end designs that remove NMS reduce latency variance and eliminate threshold-tuning overhead in production. In parallel, *exportability* (ONNX, TensorRT, CoreML, TFLite) and *quantization robustness* (FP16/INT8) increasingly determine real-world viability, as deployment often spans heterogeneous accelerators and software stacks. Within this context, the Ultralytics family illustrates a pragmatic trajectory: YOLOv5 establishes a reproducible baseline, YOLOv8 improves learning dynamics and task breadth, YOLO11 strengthens efficiency and small-object fidelity, and YOLO26 consolidates these trends with DFL removal, NMS-free decoding, and quantization-friendly graphs that scale from cloud GPUs to edge devices.

4 Deployment and Application Perspectives

The YOLO family is designed not only for research but also for practical deployment across a wide range of hardware platforms and industry applications. This section discusses how YOLOv5, YOLOv8, YOLO11, and YOLO26 are exported, quantized, and deployed on edge devices, and it surveys representative use cases in robotics, agriculture, surveillance, and manufacturing.

4.1 Export formats and hardware compatibility

Modern YOLO implementations emphasise portability [40, 10, 41]. The Ultralytics ecosystem provides export scripts that convert PyTorch checkpoints into multiple deployment formats, enabling efficient inference on diverse hardware [32]. YOLOv5 introduced a comprehensive export pipeline: models can be saved as native PyTorch weights (.pt), TorchScript (.torchscript) for embedding in C++ applications, ONNX (.onnx) for cross-framework interoperability, OpenVINO for Intel CPUs and VPUs, TensorRT (.engine) for NVIDIA GPUs, CoreML (.mlmodel) for iOS devices, TensorFlow SavedModel, GraphDef, and TFLite (.tflite) for Android and microcontroller deployment, as well as specialised formats for Edge TPUs, TensorFlow.js, and PaddlePaddle [42, 43]. Export to ONNX or OpenVINO typically yields up to three-fold acceleration on CPUs, while conversion to TensorRT can deliver five-fold GPU speedups. Table 5 summarises the major export options and the hardware targets they enable for each Ultralytics release.

Table 5: Common export formats and compatible deployment hardware. “Yes” indicates first-class support; parentheses denote community or experimental support. *Edge TPUs* refers to Google Coral and similar accelerators.

Model	PyTorch	TorchScript	ONNX / OpenVINO	TensorRT	CoreML	TFLite / TF	Edge TPUs	Other
YOLOv5	Yes	Yes	Yes	Yes	Yes	Yes	(Yes)	Paddle, TF.js
YOLOv8	Yes	Yes	Yes	Yes	Yes	Yes	(Yes)	TF.js, Paddle
YOLO11	Yes	Yes	Yes	Yes	Yes	Yes	(Yes)	TF.js, Paddle
YOLO26	Yes	(Yes)	Yes	Yes	Yes	Yes	Yes	TFLite EdgeTPU, ONNX RT

YOLOv8 refined the export interface, offering command-line flags to select formats and optimise for mixed precision (e.g. FP16). YOLO11 retained this flexibility while adding orientated bounding box support in ONNX and TensorRT exports [5]. YOLO26’s simplified head and DFL removal further streamline conversion: without distributional regression or NMS, exported graphs contain fewer custom operations and thus map more cleanly to ONNX, TensorRT, CoreML, and TFLite backends. This improved portability ensures models trained in PyTorch can be embedded into C++ pipelines, mobile applications, or web environments with minimal engineering overhead.

4.2 Quantization and edge inference

Quantization converts floating-point models into lower-precision representations to reduce memory footprint and accelerate inference [44, 45]. The YOLOv5 export pipeline supports half-precision (FP16) and 8-bit integer (INT8) quantization through ONNX Runtime, TensorRT, or TFLite. FP16 often halves memory usage and doubles throughput on compatible GPUs; INT8 can provide further gains on CPUs and NPUs, albeit with a small accuracy drop if calibration is not carefully performed. YOLOv8 inherited these capabilities and integrated mixed-precision training to ease quantization-aware deployment. YOLO11 continued to support FP16/INT8 exports while improving post-quantization accuracy by reducing depthwise convolutions and normalising weight distributions.

YOLO26 emphasises quantization robustness: by eliminating DFL and adopting an NMS-free decoder, its computational graph avoids custom operations that are difficult to quantize. Experiments show that INT8 exports of YOLO26 retain nearly the same mAP as FP32 versions, enabling smooth deployment on resource-constrained hardware such as the NVIDIA Jetson Orin and Xavier series, Qualcomm Snapdragon AI accelerators, and ARM CPUs. In addition, YOLO26 supports tensor decompositions and channel pruning, allowing further reductions in model size.

Jetson platforms are a common target for edge inference. When compiled with TensorRT in FP16 mode, YOLOv5n yields sub-2 ms latency on a T4 GPU; YOLOv8n and YOLO11n exhibit similarly low latencies with improved accuracy. For INT8 inference on Jetson Xavier NX, YOLOv8n runs at around 18–20 ms per frame while maintaining high accuracy, whereas YOLO26n can achieve comparable accuracy at lower latency owing to the simplified head. On CPUs, deploying ONNX models with OpenVINO or ONNX Runtime results in significant speedups compared with PyTorch; for example, YOLO26n runs around twice as fast as YOLOv8n at similar accuracy on x86 processors. For mobile devices, TFLite and CoreML exports enable real-time inference; YOLOv8s and YOLO26s can process 30 frames per second on modern smartphones, offering a compelling solution for on-device vision.

4.3 Applications in robotics, agriculture, surveillance and manufacturing

Robotics Real-time object detection is fundamental for autonomous navigation, manipulation, and human–robot interaction [46, 47, 48]. YOLOv5 and YOLOv8 have been widely used for robotics tasks such as obstacle avoidance, pallet detection, and warehouse inventory scanning due to their balance of speed and accuracy. YOLO11’s improved small-object detection aids precise localisation of tools or components. YOLO26’s NMS-free predictions reduce system latency and simplify integration with downstream planners; its support for pose estimation and oriented bounding boxes enables grasp planning and six-degree-of-freedom pose estimation. On mobile robots with Jetson Orin, a quantized

YOLO26n can perform multi-task perception (detection, segmentation, keypoints) within a 10–20 ms budget, permitting closed-loop control at 50–100 Hz.

Agriculture In precision agriculture, UAVs and ground robots monitor crop health, detect weeds, and estimate yield. YOLOv5 and YOLOv8 have been applied to count fruit, identify diseases, and segment weeds [49, 50]. YOLOv8’s instance segmentation head helps separate overlapping leaves, while the classification head differentiates crop species. YOLO11’s oriented detection can handle elongated objects (e.g. vines) and its improved recall benefits small buds or pests [51]. YOLO26’s STAL mechanism enhances detection of small fruits under occlusion, and its streamlined exports facilitate deployment on low-power field robots. Quantized models on embedded GPUs allow processing high-resolution aerial imagery in real time, enabling timely interventions such as targeted spraying or harvesting.

Surveillance Video surveillance systems require robust detection under varied lighting and crowded scenes [5, 52, 53]. YOLOv5 and YOLOv8 form the backbone of many security camera products, offering real-time inference on edge devices with optional cloud off-loading. YOLO11 increases detection fidelity for small objects, such as distant pedestrians, and its pose estimation module supports behavioural analysis. YOLO26’s NMS-free output simplifies pipeline design by removing threshold tuning, and its quantization robustness enables deployment on CPUs in network video recorders. Integration with segmentation tasks helps delineate intrusion zones and improve tracking; oriented bounding boxes facilitate vehicle pose estimation in traffic monitoring.

Manufacturing Automated inspection and assembly lines demand fast and accurate detection of defects and components [54, 55]. YOLOv5 and YOLOv8 have been used for defect detection in printed circuit boards, quality assessment of food products, and tool presence verification. YOLO11’s efficient architecture and improved small-object recall suit high-speed conveyors where millimetre-scale defects must be captured. Oriented bounding boxes allow proper alignment of irregular parts for robotic pick-and-place. YOLO26’s end-to-end inference and removal of DFL reduce system latency and memory, enabling deployment on programmable logic controllers and edge accelerators. Its multi-task capabilities (segmentation and classification) streamline integration with vision-guided robotic arms for assembly and packaging. Moreover, the simplified graph eases regulatory validation for safety-critical environments where deterministic execution and explainability are important.

5 Challenges and Future Directions

Advances in the YOLO family continue to push the boundaries of real-time object detection, yet several unresolved challenges remain. Dense scenes, domain adaptation, the integration of hybrid CNN–Transformer architectures, open-vocabulary detection with foundation models, and edge-aware training all represent active areas for research. This section analyses these challenges and outlines promising directions for future development.

5.1 Remaining limitations: dense scenes and domain adaptation

In crowded environments, conventional non-maximum suppression struggles to disambiguate overlapping objects, leading to missed detections and false positives [56, 57, 58]. Techniques such as Soft-NMS [59], adaptive box fusion [60], and weighted suppression [61, 62] attempt to mitigate this but require careful tuning. Anchor-free detectors improve localization granularity [63, 64], yet their performance in dense scenes still lags behind human perception. Crowd-specific datasets and synthetic data augmentation can be used to train detectors on tightly packed objects, and multi-scale feature fusion aids in recognizing small items, but occlusions remain problematic. Domain adaptation presents another major obstacle: models trained on curated datasets often generalize poorly to new domains with different lighting, textures, or sensor characteristics [65, 66]. Unsupervised domain adaptation methods, including adversarial alignment, cycle-consistent style transfer, and pseudo-label refinement, aim to bridge the gap, while meta-learning and continual learning approaches enable rapid adaptation to new environments [65]. However, stability, catastrophic forgetting, and the lack of standardized benchmarks for domain shifts hinder progress. Future work may combine explicit scene understanding with domain-aware normalization layers and self-supervised pretraining on diverse, unlabeled imagery to further close the gap.

5.2 Potential hybrid CNN–Transformer designs

Convolutional networks capture local spatial patterns efficiently, while transformers excel at modeling long-range dependencies. Hybrid CNN–Transformer architectures seek to leverage both advantages by integrating lightweight attention modules within convolutional backbones or adding transformer decoders for object queries. Early examples include networks that insert axial or bottleneck self-attention layers into residual blocks, as well as detectors that

replace parts of the neck with multi-head attention to fuse global context. Detection Transformers (DETR) introduced a transformer decoder on top of CNN features, but suffered from slow convergence; subsequent hybrids combine local convolution with sparse, hierarchical attention to balance accuracy and efficiency. Vision backbones such as CoAtNet and Next-ViT systematically merge convolutional and attention-based stages, while detection heads like HyperACE use hypergraph-enhanced self-attention to aggregate feature correlations beyond pairwise interactions. Future YOLO iterations may employ dynamic adapters that route information between CNN and transformer components based on scene complexity, and search-based design tools could automatically configure the optimal hybrid topology for a given deployment constraint. As compute resources continue to grow, hybrid architectures promise to narrow the gap between lightweight detectors and transformer-based models.

5.3 Role of open-vocabulary detection and foundation models

Traditional detectors are trained on fixed label sets, limiting their ability to recognize novel classes [67, 68, 69]. Open-vocabulary detection leverages large-scale vision–language models and foundation models to generalize beyond closed vocabularies [70]. Techniques such as CLIP, ALIGN, and BLIP align visual embeddings with language embeddings, allowing detectors to understand textual prompts and detect objects defined by arbitrary descriptions. Open-vocabulary detectors like DETIC [71] and GLIP [72] integrate contrastive pretraining with region proposal networks, enabling the detection of unseen categories by matching image regions to class names in natural language. Recent work explores zero-shot and few-shot learning, where the model infers new categories by reading class definitions or leveraging semantic hierarchies. Integration with foundation models can extend YOLO’s capabilities: the detector’s backbone can be augmented with CLIP encoders, or its classification head can be replaced with a language-conditioned module. Multimodal training regimes that combine image-caption pairs, bounding box annotations, and synthetic descriptions help the model learn generalized object concepts [73, 74]. Future research may incorporate reasoning modules that interpret compositional queries (e.g., “find red boxes on the top shelf”) and integrate with language models for interactive detection. Challenges include ensuring the reliability of language-conditioned responses, handling ambiguous descriptions, and maintaining real-time performance despite the added complexity.

5.4 Edge-aware training and hardware-in-the-loop optimization

Deploying detectors on edge devices requires careful co-design of algorithms and hardware. Quantization-aware training and mixed-precision strategies reduce memory and computational requirements while preserving accuracy. Channel pruning and neural architecture search tailor the network topology to target processors, but static optimization may not account for runtime variation. Hardware-in-the-loop optimization addresses this by incorporating feedback from the deployment environment during training: latency, power consumption, and temperature measurements guide the search towards architectures that meet constraints. Frameworks such as Once-For-All allow a single supernet to be specialized dynamically for different devices, while reinforcement learning approaches train controllers to adjust network width, depth, or input resolution based on real-time feedback. Edge-aware training extends this concept by incorporating device-specific operations (e.g., tensor cores, DSPs) directly into the computational graph, ensuring that gradient updates reflect actual hardware behavior. Continual profiling and adaptive quantization can maintain efficiency as hardware or workload conditions change. For example, a model running on a Jetson device might reduce its precision or skip layers when battery levels are low, trading accuracy for energy savings. Future research may explore co-optimization across the entire pipeline from data acquisition and preprocessing to neural inference and post-processing to achieve robust, high-throughput deployment in heterogeneous edge environments.

6 Acknowledgment

This work was supported in part by the National Science Foundation (NSF) and the United States Department of Agriculture (USDA), National Institute of Food and Agriculture (NIFA), through the “Artificial Intelligence (AI) Institute for Agriculture” program under Award Numbers AWD003473 and AWD004595, and USDA-NIFA Accession Number 1029004 for the project titled “Robotic Blossom Thinning with Soft Manipulators.” Additional support was provided through USDA-NIFA Grant Number 2024-67022-41788, Accession Number 1031712, under the project “ExPanding UCF AI Research To Novel Agricultural Engineering Applications (PARTNER).”

References

- [1] Zhong-Qiu Zhao, Peng Zheng, Shou-tao Xu, and Xindong Wu. Object detection with deep learning: A review. *IEEE transactions on neural networks and learning systems*, 30(11):3212–3232, 2019.

- [2] Zhengxia Zou, Keyan Chen, Zhenwei Shi, Yuhong Guo, and Jieping Ye. Object detection in 20 years: A survey. *Proceedings of the IEEE*, 111(3):257–276, 2023.
- [3] Chhavi Rana et al. Artificial intelligence based object detection and traffic prediction by autonomous vehicles—a review. *Expert Systems with Applications*, 255:124664, 2024.
- [4] Zohaib Khan, Yue Shen, and Hui Liu. Object detection in agriculture: A comprehensive review of methods, applications, challenges, and future directions. *Agriculture*, 15(13):1351, 2025.
- [5] Ranjan Sapkota, Marco Flores-Calero, Rizwan Qureshi, Chetan Badgajar, Upesh Nepal, Alwin Poullose, Peter Zeno, Uday Bhanu Prakash Vaddevolu, Sheheryar Khan, Maged Shoman, et al. Yolo advances to its genesis: a decadal and comprehensive review of the you only look once (yolo) series. *Artificial Intelligence Review*, 58(9):274, 2025.
- [6] Ranjan Sapkota, Rahul Harsha Cheppally, Ajay Sharda, and Manoj Karkee. Yolo26: key architectural enhancements and performance benchmarking for real-time object detection. *arXiv preprint arXiv:2509.25164*, 2025.
- [7] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [8] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.
- [9] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE transactions on pattern analysis and machine intelligence*, 39(6):1137–1149, 2016.
- [10] Momina Liaqat Ali and Zhou Zhang. The yolo framework: A comprehensive review of evolution, applications, and benchmarks in object detection. *Computers*, 13(12):336, 2024.
- [11] Tausif Diwan, G Anirudh, and Jitendra V Tembhurne. Object detection using yolo: challenges, architectural successors, datasets and applications. *multimedia Tools and Applications*, 82(6):9243–9275, 2023.
- [12] Joseph Redmon and Ali Farhadi. Yolo9000: better, faster, stronger. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7263–7271, 2017.
- [13] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [14] Kyriakos D Apostolidis and George A Papakostas. Delving into yolo object detection models: Insights into adversarial robustness. *Electronics*, 14(8):1624, 2025.
- [15] Enerst Edozie, Aliyu Nuhu Shuaibu, Ukagwu Kelechi John, and Bashir Olaniyi Sadiq. Comprehensive review of recent developments in visual object detection based on deep learning. *Artificial Intelligence Review*, 58(9):277, 2025.
- [16] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.
- [17] Chuyi Li, Lulu Li, Hongliang Jiang, Kaiheng Weng, Yifei Geng, Liang Li, Zaidan Ke, Qingyuan Li, Meng Cheng, Weiqiang Nie, et al. Yolov6: A single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022.
- [18] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 7464–7475, 2023.
- [19] Mupparaju Sohan, Thotakura Sai Ram, and Ch Venkata Rami Reddy. A review on yolov8 and its advancements. In *International Conference on Data Intelligence and Cognitive Informatics*, pages 529–545. Springer, 2024.
- [20] Javaria Farooq, Muhammad Muaz, Khurram Khan Jadoon, Nayyer Aafaq, and Muhammad Khizer Ali Khan. An improved yolov8 for foreign object debris detection with optimized architecture for small objects. *Multimedia Tools and Applications*, 83(21):60921–60947, 2024.
- [21] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9: Learning what you want to learn using programmable gradient information. In *European conference on computer vision*, pages 1–21. Springer, 2024.
- [22] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, et al. Yolov10: Real-time end-to-end object detection. *Advances in Neural Information Processing Systems*, 37:107984–108011, 2024.
- [23] Yunjie Tian, Qixiang Ye, and David Doermann. Yolov12: Attention-centric real-time object detectors. *arXiv preprint arXiv:2502.12524*, 2025.

- [24] Mengqi Lei, Siqi Li, Yihong Wu, Han Hu, You Zhou, Xinhua Zheng, Guiguang Ding, Shaoyi Du, Zongze Wu, and Yue Gao. Yolov13: Real-time object detection with hypergraph-enhanced adaptive visual perception. *arXiv preprint arXiv:2506.17733*, 2025.
- [25] Chen Fang and Xiang Yang. Lightweight yolov8 for wheat head detection. *IEEE Access*, 12:66214–66222, 2024.
- [26] Lei Liang and Junming Chen. Vst-yolov8: A trustworthy and secure defect detection framework for industrial gaskets. *Electronics*, 14(19):3760, 2025.
- [27] Parya Dolatyabi, Jacob Regan, and Mahdi Khodayar. Deep learning for traffic scene understanding: A review. *IEEE Access*, 2025.
- [28] Jiaxue Liu, Chao Zhang, and Jianjun Li. Using anchor-free object detectors to detect surface defects. *Processes*, 12(12):2817, 2024.
- [29] Yan Zhang, Haifeng Zhang, Qingqing Huang, Yan Han, and Minghang Zhao. Dsp-yolo: An anchor-free network with dspan for small object detection of multiscale defects. *Expert Systems with Applications*, 241:122669, 2024.
- [30] Xueqiu Wang, Huanbing Gao, Zemeng Jia, and Zijian Li. Bl-yolov8: An improved road defect detection model based on yolov8. *Sensors*, 23(20):8361, 2023.
- [31] Guoliang Yang, Jixiang Wang, Ziling Nie, Hao Yang, and Shuaiying Yu. A lightweight yolov8 tomato detection algorithm combining feature enhancement and attention. *Agronomy*, 13(7):1824, 2023.
- [32] Glenn Jocher, Ayush Chaurasia, Alex Stoken, Jirka Borovec, Yonghye Kwon, Kalen Michael, Jiacong Fang, Colin Wong, Zeng Yifu, Diego Montes, et al. ultralytics/yolov5: v6. 2-yolov5 classification models, apple m1, reproducibility, clearml and deci. ai integrations. *Zenodo*, 2022.
- [33] Jian Ren, Zhijie Wang, Yifan Zhang, and Lei Liao. Yolov5-r: lightweight real-time detection based on improved yolov5. *Journal of Electronic Imaging*, 31(3):033033–033033, 2022.
- [34] Chao Dang, Zaixing Wang, Yonghuan He, Linchang Wang, Yi Cai, Huiji Shi, and Jiachi Jiang. The accelerated inference of a novel optimized yolov5-lite on low-power devices for railway track damage detection. *IEEE Access*, 11:134846–134865, 2023.
- [35] Glenn Jocher, Alex Stoken, Jirka Borovec, Liu Changyu, Adam Hogan, Ayush Chaurasia, Laurentiu Diaconu, Francisco Ingham, Adrien Colmagro, Hu Ye, et al. ultralytics/yolov5: v4. 0-nn. silu () activations, weights & biases logging, pytorch hub integration. *Zenodo*, 2021.
- [36] Richard Johnson. *YOLO Object Detection Explained: Definitive Reference for Developers and Engineers*. HiTeX Press, 2025.
- [37] Emmanuel C Nnadozie, Pablo Casaseca-de-la Higuera, Ogechukwu Iloanusi, Ozoemena Ani, and Carlos Alberola-López. Simplifying yolov5 for deployment in a real crop monitoring setting. *Multimedia Tools and Applications*, 83(17):50197–50223, 2024.
- [38] Jiacong Fang, Qiong Liu, and Jingzheng Li. A deployment scheme of yolov5 with inference optimizations based on the triton inference server. In *2021 IEEE 6th International Conference on cloud computing and big data analytics (ICCCBDA)*, pages 441–445. IEEE, 2021.
- [39] Bin Yan, Pan Fan, Xiaoyan Lei, Zhijie Liu, and Fuzeng Yang. A real-time apple targets detection method for picking robot based on improved yolov5. *Remote Sensing*, 13(9):1619, 2021.
- [40] Dan Munteanu, Diana Moina, Cristina Gabriela Zamfir, Ștefan Mihai Petrea, Dragos Sebastian Cristea, and Nicoleta Munteanu. Sea mine detection framework using yolo, ssd and efficientdet deep learning models. *Sensors*, 22(23):9536, 2022.
- [41] Partha Pratim Ray, Mohan Pratap Pradhan, and Shuai Li. Llmyoloedge: An edge-iot aware novel framework for integration of yolo with localized quantized large language models. *IEEE Access*, 2025.
- [42] Glenn Jocher, Alex Stoken, Jirka Borovec, Liu Changyu, Adam Hogan, Laurentiu Diaconu, Jake Poznanski, Lijun Yu, Prashant Rai, Russ Ferriday, et al. ultralytics/yolov5: v3. 0. *Zenodo*, 2020.
- [43] Jun-Hwa Kim, Namho Kim, Yong Woon Park, and Chee Sun Won. Object detection and classification based on yolo-v5 with improved maritime dataset. *Journal of Marine Science and Engineering*, 10(3):377, 2022.
- [44] Chen Wu, Mingyu Wang, Xinyuan Chu, Kun Wang, and Lei He. Low-precision floating-point arithmetic for high-performance fpga-based cnn acceleration. *ACM Transactions on Reconfigurable Technology and Systems (TRETs)*, 15(1):1–21, 2021.
- [45] Penghao Xiao, Chunjie Zhang, Qian Guo, Xiayang Xiao, and Haipeng Wang. Neural networks integer computation: Quantizing convolutional neural networks of inference and training for object detection in embedded systems. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2024.

- [46] Sanzhar Rakhimkul, Anton Kim, Askarbek Pazylbekov, and Almas Shintemirov. Autonomous object detection and grasping using deep learning for design of an intelligent assistive robot manipulation system. In *2019 IEEE International Conference on Systems, Man and Cybernetics (SMC)*, pages 3962–3968. IEEE, 2019.
- [47] Nicole Robinson, Brendan Tidd, Dylan Campbell, Dana Kulić, and Peter Corke. Robotic vision for human-robot interaction and collaboration: A survey and systematic review. *ACM Transactions on Human-Robot Interaction*, 12(1):1–66, 2023.
- [48] Rajesh Kannan Megalingam, Vignesh S Naick, Manaswini Motheram, Jahnavi Yannam, Nikhil Chowdary Gutlapalli, and Vinu Sivanantham. Robot operating system based autonomous navigation platform with human robot interaction. *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, 21(3):675–683, 2023.
- [49] Chetan M Badgajar, Alwin Poullose, and Hao Gan. Agricultural object detection with you only look once (yolo) algorithm: A bibliometric and systematic literature review. *Computers and Electronics in Agriculture*, 223:109090, 2024.
- [50] Tomáš Zoubek, Roman Bumbálek, Jean de Dieu Marcel Uftikirezi, Miroslav Strob, Martin Filip, František Špalek, Aleš Heřmánek, and Petr Bartoš. Advancing precision agriculture with computer vision: A comparative study of yolo models for weed and crop recognition. *Crop Protection*, 190:107076, 2025.
- [51] Qiufang Dai, Shiyao Liang, Zhen Li, Shilei Lyu, Xiuyun Xue, Shuran Song, Ying Huang, Shaoyu Zhang, and Jiaheng Fu. Yolov11-rdtnet: A lightweight model for citrus pest and disease identification based on an improved yolov11n. *Agronomy*, 15(5):1252, 2025.
- [52] Huy Hoang Nguyen, Thi Nhung Ta, Ngoc Cuong Nguyen, Van Truong Bui, Hung Manh Pham, and Duc Minh Nguyen. Yolo based real-time human detection for smart video surveillance at the edge. In *2020 IEEE eighth international conference on communications and electronics (ICCE)*, pages 439–444. IEEE, 2021.
- [53] Salama A Mostafa, Sharran Ravi, Dilovan Asaad Zebari, Nechirvan Asaad Zebari, Mazin Abed Mohammed, Jan Nedoma, Radek Martinek, Muhammet Deveci, and Weiping Ding. A yolo-based deep learning model for real-time face mask detection via drone surveillance in public spaces. *Information Sciences*, 676:120865, 2024.
- [54] Dinh-Lam Pham, Tai-Woo Chang, et al. A yolo-based real-time packaging defect detection system. *Procedia Computer Science*, 217:886–894, 2023.
- [55] Wei-Lung Mao, Chun-Chi Wang, Po-Heng Chou, and Yen-Ting Liu. Automated defect detection for mass-produced electronic components based on yolo object detection models. *IEEE Sensors Journal*, 24(16):26877–26888, 2024.
- [56] Md Tanzib Hosain, Asif Zaman, Mushfiqur Rahman Abir, Shanjida Akter, Sawon Mursalin, and Shadman Sakeeb Khan. Synchronizing object detection: Applications, advancements and existing challenges. *IEEE access*, 12:54129–54167, 2024.
- [57] Mahya Nikouei, Bitá Baroutian, Shahabedin Nabavi, Fateme Taraghi, Atefe Aghaei, Ayoob Sajedi, and Mohsen Ebrahimi Moghaddam. Small object detection: A comprehensive survey on challenges, techniques and real-world applications. *arXiv preprint arXiv:2503.20516*, 2025.
- [58] Yu Shao, Jianhua Hu, Lihua Hu, Jifu Zhang, and Xinbo Wang. Context feature fusion and enhanced non-maximum suppression for pedestrian detection in crowded scenes. *Multimedia Tools and Applications*, 84(8):4093–4113, 2025.
- [59] Navaneeth Bodla, Bharat Singh, Rama Chellappa, and Larry S Davis. Soft-nms—improving object detection with one line of code. In *Proceedings of the IEEE international conference on computer vision*, pages 5561–5569, 2017.
- [60] Oier Mees, Andreas Eitel, and Wolfram Burgard. Choosing smartly: Adaptive multimodal fusion for object detection in changing environments. In *2016 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 151–156. IEEE, 2016.
- [61] Yu Wang, Rui Zhang, Shuo Zhang, Miao Li, YangYang Xia, XiShan Zhang, and ShaoLi Liu. Domain-specific suppression for adaptive object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9603–9612, 2021.
- [62] Xiaoqi Zhao, Youwei Pang, Lihe Zhang, Huchuan Lu, and Lei Zhang. Suppress and balance: A simple gated network for salient object detection. In *European conference on computer vision*, pages 35–51. Springer, 2020.
- [63] Ying Xin, Guanzhong Wang, Mingyuan Mao, Yuan Feng, Qingqing Dang, Yanjun Ma, Errui Ding, and Shumin Han. Pafnet: An efficient anchor-free object detector guidance. *arXiv preprint arXiv:2104.13534*, 2021.
- [64] Zhimin Tang, Yuhui Deng, Yi Zhou, Hexian Lu, Lijuan Lu, Junhao Huang, Hong Li, and Shun Long. Gpdet: an anchor-free object detector based on dual center-ness and criss-cross balance for unstructured gastroscopic image data. *The Journal of Supercomputing*, 81(10):1–29, 2025.

- [65] Poojan Oza, Vishwanath A Sindagi, Vishal M Patel, et al. Unsupervised domain adaptation of object detectors: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(6):4018–4040, 2023.
- [66] Peeyush Singhal, Rahee Walambe, Sheela Ramanna, and Ketan Kotecha. Domain adaptation: challenges, methods, datasets, and applications. *IEEE access*, 11:6973–7020, 2023.
- [67] Pramuditha Perera and Vishal M Patel. Deep transfer learning for multiple class novelty detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11544–11552, 2019.
- [68] Prannay Kaul, Weidi Xie, and Andrew Zisserman. Label, verify, correct: A simple few shot object detection method. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14237–14247, 2022.
- [69] Paul Bodesheim, Alexander Freytag, Erik Rodner, and Joachim Denzler. Local novelty detection in multi-class recognition problems. In *2015 IEEE Winter Conference on Applications of Computer Vision*, pages 813–820. IEEE, 2015.
- [70] Jianzong Wu, Xiangtai Li, Shilin Xu, Haobo Yuan, Henghui Ding, Yibo Yang, Xia Li, Jiangning Zhang, Yunhai Tong, Xudong Jiang, et al. Towards open vocabulary learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(7):5092–5113, 2024.
- [71] Xingyi Zhou, Rohit Girdhar, Armand Joulin, Philipp Krähenbühl, and Ishan Misra. Detecting twenty-thousand classes using image-level supervision. In *European conference on computer vision*, pages 350–368. Springer, 2022.
- [72] Liunian Harold Li, Pengchuan Zhang, Haotian Zhang, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, et al. Grounded language-image pre-training. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10965–10975, 2022.
- [73] Xinjie Zhang, Jintao Guo, Shanshan Zhao, Minghao Fu, Lunhao Duan, Jiakui Hu, Yong Xien Chng, Guo-Hua Wang, Qing-Guo Chen, Zhao Xu, et al. Unified multimodal understanding and generation models: Advances, challenges, and opportunities. *arXiv preprint arXiv:2505.02567*, 2025.
- [74] Iro Laina, Christian Rupprecht, and Nassir Navab. Towards unsupervised image captioning with shared multimodal embeddings. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 7414–7424, 2019.